

1.4.1

digital thread trace of product lifecycle

digital twin real-time system virtual replica

1.5

supervised regression, classification, prediction, ...

unsupervised clustering, dimensionality reduction, ...

semi-supervised clustering 聚类 → labelling

2

terminology

features - input, $x^{(i)} = (1, x_1^{(i)}, x_2^{(i)}, \dots, x_N^{(i)})$ * $x = x^T$

* e.g. $x^{(i)} = (1, x_1^{(i)}, x_2^{(i)}, \dots, x_N^{(i)})^T$

label/target - i -th data point of the feature, $y^{(i)}$

training set - $y = (y^{(1)}, y^{(2)}, \dots, y^{(m)})^T$ * classes in classification

bias - θ_0

weights - θ_i

parameters - $\theta_{(n)} = (\theta_0, \theta_1, \dots, \theta_N)_{(n)}$ * $\theta_{(n)} = \theta_{(n)}^T$

hypothesis function - $h(x^{(i)}) = \theta \cdot x^{(i)} = \theta_0 + \theta_1 x_1^{(i)} + \dots$

loss function - $J_{(n)}(\theta)$

learning rate - α

regularization parameter - λ

epoch - how many times let model learn from data

convex - local = global minimum, $f(y) \geq f(x) + \nabla f(x) \cdot (y - x)$

* e.g. $\exp(sx), s \in \mathbb{R}, x^2, n \in \mathbb{Z}, 4, \dots, -\ln(x), x > 0, \frac{1}{x}, x, r > 0$

loss function convex for linear $h(x)$

mean-squared error (mse) - $\frac{1}{2m} \sum_{i=1}^m (h(x_k^{(i)}) - y^{(i)})^2$

mean absolute error (mae) - $\frac{1}{2m} \sum_{i=1}^m |h(x_k^{(i)}) - y^{(i)}|$

Huber loss - mse for small and mae for large errors * prefer in 6

1.3 - 1.5

multi-variable linear regression

1. choose starting point $\theta_{(0)} = (\theta_0, \theta_1, \dots, \theta_N)_{(0)}$

2. calculate gradient $\nabla_{\theta} J_{(0)} = (\frac{\partial J}{\partial \theta_0}, \frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_N})_{(0)}^T$

* vectorization $\nabla_{\theta} J = \frac{1}{m} X^T (X\theta - y), X = (x^{(1)}, x^{(2)}, \dots, x^{(m)})^T$

3. update $\theta_{(1)} = \theta_{(0)} - \alpha \nabla_{\theta} J_{(0)}$

4. repeats until converge to tolerance or reach maximum iter.

1.6

regularization reduce test error * strictly convex

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h(x_k^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^N \theta_j^2$$

1.7.2

feature scaling magnitude of x_1 much larger than x_2

$x_j = \frac{x_j - \mu_j}{\sigma_j}$ * also good for logistic regression, SVM, PCA, k-NN, k-means, DBSCAN...

1.8

batch gradient descent use whole set per iter

* prone to overfitting and have memory requirement

stochastic 随机的 - use a random data point per iter

mini-batch - use a small batch of size m_b per iter

Made with Goodnotes

easily parallelized in a computer software

3

1.2

logistic regression uses probability to do binary classification

sigmoid uncertainty of the feature vector belongs to a set (class)

$$\sigma(x) = h(y = 1|x) = \frac{1}{1 + e^{-\theta \cdot x}} = \frac{1}{1 + e^{-\theta_0 - \theta_1 x_1 - \dots - \theta_N x_N}} \in (0, 1)$$

$$\frac{\partial}{\partial x} \sigma(x) = \sigma(x)(1 - \sigma(x))$$

1.3

(average) cross-entropy loss function convex for $h(x) = \sigma(x)$

$$J(\theta) = J_{y^{(i)}=1} + J_{y^{(i)}=0} = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \ln[h(x^{(i)})] + (1 - y^{(i)}) \ln[1 - h(x^{(i)})]]$$

$$\nabla_{\theta} J = \frac{1}{m} X^T (\sigma(X\theta) - y), \text{ need to apply } \sigma \text{ to } X\theta \text{ elementwise}$$

1.4

logistic regularization same as regularization * strictly convex

1.5

1.6

multiclass problems one-vs-all

n. of logistic regression classifiers needed = n. class

1.7

confusion matrix

Model Prediction $\sigma(x) = h(x)$	Ground Truth $y^{(i)}$		Precision $\frac{TP}{TP+FP}$
	1	0	
1	True Positive	False Positive	
0	False Negative	True Negative	

Accuracy $\frac{TP+TN}{TP+FP+FN+TN}$	Recall $\frac{TP}{TP+FN}$	F-score 2 $\frac{Precision \times Recall}{Precision + Recall}$
--------------------------------------	---------------------------	--

$$\text{ideally } = 1$$

2.3

hyperplane $w \cdot x + b = \text{orthogonal} + \text{shift} = w \cdot (x - x_0) = 0$

distance of x_B from hyperplane is $M = \frac{f(x_B)}{\|w\|} = \frac{w \cdot x_B + b}{\|w\|} = \frac{w \cdot (x_1 + Mw/\|w\|) + b}{\|w\|}$

2.4

support vector machine (SVM) when n.features > n.examples

classification - find maximum-margin separation hyperplane, maximize $\frac{2}{\|w\|}$

* normalize $w \rightarrow \frac{w}{M/\|w\|}$, maximize $\frac{2}{\|w\|} = \text{minimize } \frac{1}{2} \|w\|^2$

2.4.2

regularization and soft margins * strictly convex

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

$$\text{subject to } y^{(i)}(w \cdot x^{(i)} + b) \geq 1 - \xi_i, \text{ for } \xi_i \geq 0$$

* small C - soft (wide) margin, C $\rightarrow \infty$ - hard margin

2.5

nonlinear separable classes feature mapping

enriching the data - $f(x^{(i)}) = w \cdot \Phi(x^{(i)}) + b$

* e.g. $\Phi(x_1^{(i)}, x_2^{(i)}) = (x_1^{(i)}, x_2^{(i)}, x_1^{(i)2} + x_2^{(i)2})$

* computationally expensive by adding di explicitly

kernel trick - $f(x^{(i)}) = \sum_{j=1}^m \beta_j \Phi(x^{(j)}) \cdot \Phi(x^{(i)}) + b$

$= \sum_{j=1}^m \beta_j K(x^{(j)}, x^{(i)}) + b$

* e.g. polynomial, $K(y \cdot x) = (y \cdot x + 1)^2$

radial basis function, $K(y \cdot x) = \exp(-\gamma \|y - x\|^2)$

* enables higher-dimensional feature space without cost

4

1.3 - 1.5

nonlinear function linear combination of linear function

shift - rotate - add up - deform - repeat

one neuron

feedforward neural networks

Input layer Hidden layer Hidden layer Output layer

input layer - $x^{(i)} = (x_0^{(i)}, x_1^{(i)}, x_2^{(i)}, \dots, x_N^{(i)})^T$ * $x_0^{(i)} = 1$

1st hidden layer -

$\theta^{(0)} = \begin{pmatrix} \theta_{0,1}^{(0)} & \theta_{0,2}^{(0)} & \theta_{0,3}^{(0)} \\ \theta_{1,1}^{(0)} & \theta_{1,2}^{(0)} & \theta_{1,3}^{(0)} \\ \vdots & \vdots & \vdots \\ \theta_{N,1}^{(0)} & \theta_{N,2}^{(0)} & \theta_{N,3}^{(0)} \end{pmatrix}, z^1 = \begin{pmatrix} b_0^1 \\ \theta^{(0)T} x^1 \end{pmatrix}, a^1 = a(z^1) = \begin{pmatrix} a_0^1 \\ a_1^1 \\ \vdots \\ a_N^1 \end{pmatrix}$

2nd hidden layer -

$\theta^{(1)} = \begin{pmatrix} \theta_{0,1}^{(1)} & \theta_{0,2}^{(1)} & \theta_{0,3}^{(1)} \\ \theta_{1,1}^{(1)} & \theta_{1,2}^{(1)} & \theta_{1,3}^{(1)} \\ \vdots & \vdots & \vdots \\ \theta_{N,1}^{(1)} & \theta_{N,2}^{(1)} & \theta_{N,3}^{(1)} \end{pmatrix}, z^2 = \begin{pmatrix} b_0^2 \\ \theta^{(1)T} a^1 \end{pmatrix}, a^2 = a(z^2)$

output layer -

$\theta^{(2)} = \begin{pmatrix} \theta_{0,1}^{(2)} & \theta_{0,2}^{(2)} \\ \theta_{1,1}^{(2)} & \theta_{1,2}^{(2)} \\ \vdots & \vdots \\ \theta_{N,1}^{(2)} & \theta_{N,2}^{(2)} \end{pmatrix}, z^3 = \begin{pmatrix} b_0^3 \\ \theta^{(2)T} a^2 \end{pmatrix}, h = a(z^3)$

1.8

activation function all a below are monotonic increasing

* non-differentiable at z = 0

* may suffer from vanishing gradient at large |z|

1.8

output layer activation function varies with task

regression - linear units * $J = \text{'MeanSquaredError'}$

binary classification [0, 1] - sigmoid * $J = \text{'BinaryCrossentropy'}$

multi-class classification [0, 1, ..., k] - softmax

* softmax(z_i) = $\frac{\exp(z_i)}{\sum_j \exp(z_j)}$ = probability of feature belongs class i

Label Encoding			One Hot Encoding			
Food Name	Categorical	Calories	Apple	Chicken	Broccoli	Calories
Apple	1	95	1	0	0	95
Chicken	2	231	0	1	0	231
Broccoli	3	50	0	0	1	50

* SparseCategoricalCrossentropy()

* CategoricalCrossentropy()

1.6

number of parameters

n. of paramaters = $\sum_{i=0}^m (s_i + 1) s_{i+1}$ * for input layer, $s_0 = N$

1.7

forward propagation evaluate loss function

regression -

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \sum_{k=1}^K [h_k(x^{(i)}) - y_k^{(i)}]^2 + \lambda \sum_{i=0}^L \sum_{a=0}^{s_i} \sum_{b=0}^{s_{i+1}} (\theta_{a,b}^{(i)})^2$$

classification -

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K [y_k^{(i)} \ln[h_k(x^{(i)})] + (1 - y_k^{(i)}) \ln[1 - h_k(x^{(i)})]] + \lambda \sum_{i=0}^L \sum_{a=0}^{s_i} \sum_{b=0}^{s_{i+1}} (\theta_{a,b}^{(i)})^2$$

* K = n. of output

* loss function may non-convex, very likely local $\neq$ global minimum

1.10

backpropagation evaluate gradient of loss function

chain rule -

$$\frac{dJ}{d\theta^{(l)}} = \frac{dJ}{dh} \frac{dh}{da^{l+1}} \frac{da^{l+1}}{d\theta^{(l)}} = \frac{dJ}{dh} \frac{dh}{da^{l+1}} \frac{da^{l+1}}{dz^{l+1}} \frac{dz^{l+1}}{da^l} \frac{da^l}{dz^l} \frac{dz^l}{d\theta^{(l)}}$$

$$+ \frac{dz^{l+1}}{da^l} = \frac{d\theta^{(l)T} a^l}{da^l} = \theta^{(l)T}, \frac{da^l}{dz^l} = \frac{da(z^l)}{dz^l} = a'(z^l)$$

backpropagation -

$$\frac{dh}{d\theta_{a,b}^{(l)}} = \frac{dh}{dz^{l+1}} \frac{dz^{l+1}}{d\theta_{a,b}^{(l)}} = \delta_b^{l+1} a^l$$

* δ is error at layer l, $\delta_j^l = (\cdot)'(z_j^l)$ in $\delta^{l+1} \theta^{(l)T} a^{l-1}(z^l)$

* involves only multiplication operations as forward store in memory

1.15

TensorFlow notation Python start from 0 instead of 1

input layer - $x = (x_0, x_1, x_2, \dots, x_{N-1})$

l-th hidden layer -

$\theta^{(l)} = \begin{pmatrix} \theta_{0,0}^{(l)} & \dots & \theta_{0,n_l-1}^{(l)} \\ \theta_{1,0}^{(l)} & \dots & \theta_{1,n_l-1}^{(l)} \\ \vdots & \ddots & \vdots \\ \theta_{n_{l-1}-1,0}^{(l)} & \dots & \theta_{n_{l-1}-1,n_l-1}^{(l)} \end{pmatrix}$ * $n_l = \text{n. of node in layer}$

$b^{(l)} = (b_1^{(l)}, b_2^{(l)}, \dots, b_{N^{(l)}}^{(l)})$, $z^l = a^{l-1} \theta^{(l)} + b^{(l)}$, $a^l = a(z^l)$

1.11

architecture design

fine tuning - finding good set of hyperparameters

n. of nodes - prefer same number of neurons in all hidden layers

n. of layers - n. of neurons exponentially related to n. of layers

* deep learning repeat composition functions (layers) to reduce n. of required neurons by a large factor, but often harder to train

* always start with more layers and nodes than actually need

1.12

training initialize - 1.7 - 1.10 - gradient descent

shuffle 打乱 the data randomly * buffer size > data size

split data in training (60-80%), validation (10-20%) and test (10-20%) * test set never use to train model

optimizer

standard gradient descent (sgd) - fixed learning rate α

* may get stuck in local minima

AdaGrad - adaptive α , accumulate all past gradients

* learning rate decrease fast to small values

AdaDelta - adaptive α based on moving window

RMSProp - adaptive α using exponential moving avg.

Adam - RMSProp + momentum * fast and robust

* Nesterov momentum gradient descent, β typically 0.9

$$\theta_{(n+1)} = \theta_{(n)} - \alpha [(1 - \beta) \nabla_{\theta} J_{(n)} + \beta \nabla_{\theta} J_{(n-1)}]$$

5

1.3

CNNs convolutional neural networks

* fully-connected NN contain too many parameters for grid-like data, such as images

* spital 空间的 dependencies are lost in 4

CNN arrange neurons in 3D - width $\times$ height $\times$ depth

1.4

filter weight average of pixels by convolution

kernel 2D slice of 3D filter

$$\begin{bmatrix} 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

input image raw pixel values * depth=3 for RGB

convolution layer set of learnable filters

stride S - step = resolution

stride: 1

stride: 2

padding: 0

padding P - margin

padding: 1

number of neurons - in direction $d = x, y$

* input length W_d , filter width F_d

$$\frac{W_d - F_d + 2P_d}{S_d} + 1$$

receptive field of the neuron - 3D region in one conv.

* spatially local along filter w & h , fully along input depth

depth K - n. of filters = n. of feature (activation) maps

* in direction z

input depth = filter depth

= n. of channels

output depth = n. of filters

parameter sharing - filter's param. fixed in one iter.

number of parameters - in layer l

$$(F_x \times F_y \times \text{n. of channels} + 1) \times \text{n. filters}$$

activation layer introduce nonlinearity, typically ReLU

- * do not change size of the volume (w × h × depth)
- * part of the convolution layer in theory, but explicit in TensorFlow

output after convolution (and activation) layer → **feature map** in layer l

$$z_{l,j,k} = a \left(\sum_{u=0}^{F_x-1} \sum_{v=0}^{F_y-1} \sum_{w=0}^{F_z-1} b_k + x_{(l s_x+u), (l s_y+v), (l s_z+w)} \cdot \theta_{u,v,w} \right)$$

pooling layer reduce the spatial size, no trainable param.

- * max-pooling discarding 75% of act. layer with $F = 2, S = 2, P = 1$
- * start with double the n. of filters after each pooling

flatten layer → **fully-connected layer** feedforward NN 4

1.5

augment data **regularization** prevent overfitting

artificially increase the data size by generating realistic variations of the training data points - **shift, rotate, resize, contrast, ...**

dropout **regularization** prevent overfitting

dropout $p\%$ neurons at every training step except output layer

- * dropout rate p typically set to 40 – 50%

coding TensorFlow

tf.keras.layers.Conv(n, filters, (filter volume), activation, input_shape)

- * default setting gives strides (1, 1) and padding 'valid' → $P = 0$

6 p3–p13

sequential data both data and **order** contain information

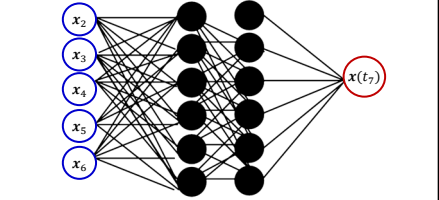
speech recognition, music generation, sentiment 观点 classification, DNA sequence analysis, machine translation, name recognition, video activity classification, **time series, dynamical systems, sensor...**

sigmoid uncertainty of the feature vector belongs to a set (class)

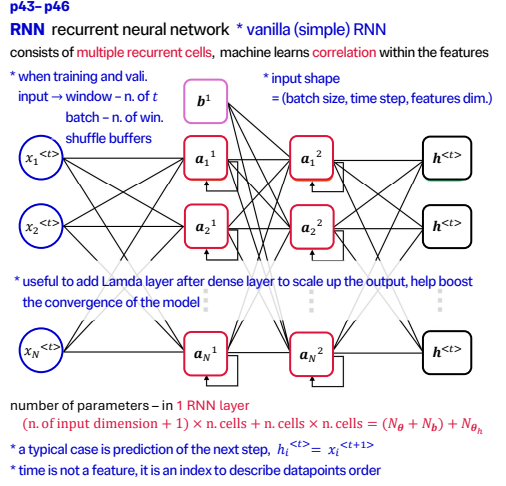
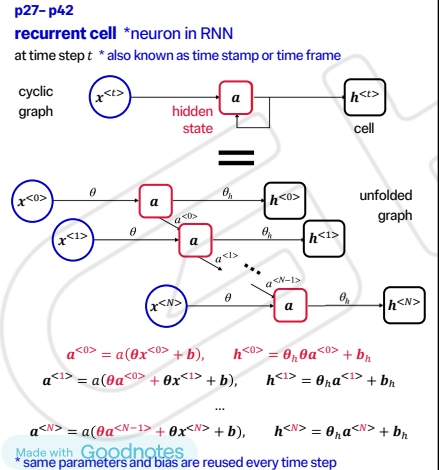
p14–p26

sequence modelling with feedforward NN 4

typically not good approach - **can only retain memory for short past**



- * do not learn correlation in time, interpolate instead
- * has large parameters
- * need inputs of fixed size



p47–p57

loss function sum of the loss function of **every timestep**

$$J(\theta) = \sum_{t=0}^N J^{(t)}$$

* unlike training set in feedforward NN, which start from (1), time start from $< 0 >$

backpropagation in time

chain rule - * assume $a = 1$ for brevity

$$\frac{\partial J}{\partial a \langle t \rangle} = \frac{\partial J}{\partial a \langle N \rangle} \frac{\partial a \langle N \rangle}{\partial a \langle t \rangle} = \frac{\partial J}{\partial a \langle N \rangle} \frac{\partial a \langle N \rangle}{\partial a \langle N-1 \rangle} \dots \frac{\partial a \langle t+1 \rangle}{\partial a \langle t \rangle} = \frac{\partial J}{\partial a \langle N \rangle} (\theta)^{N-t}$$

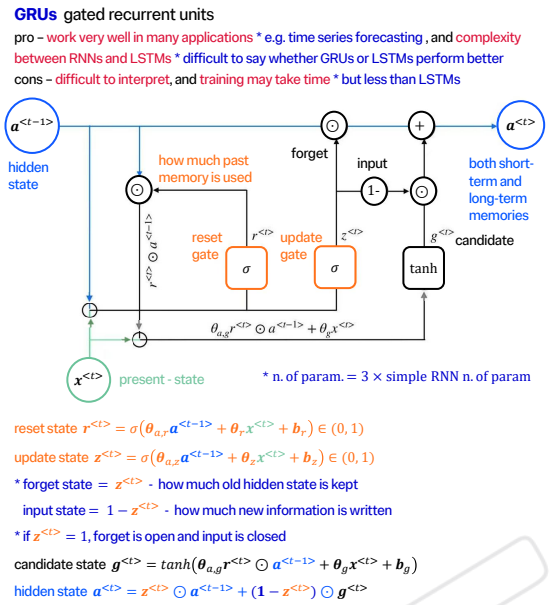
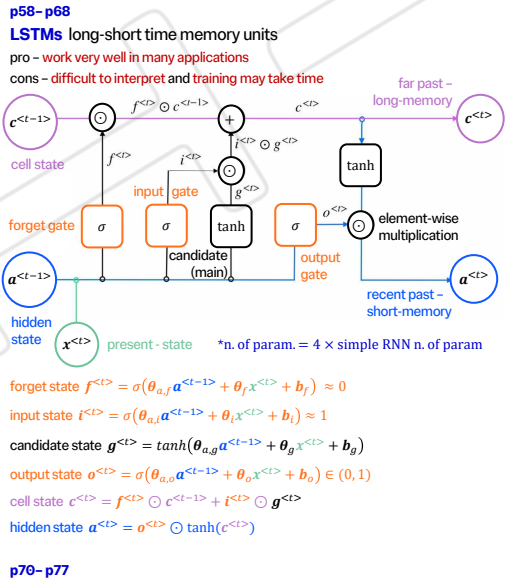
vanishing gradient - if θ is too small, $N - 1$ times θ will push next iteration θ become even smaller

- * this can be solved
- 1. use ReLU - unbounded +ve, gradient will not decay with large input
- 2. use residual learning - skip connections, carry information through deep layers
- 3. move to LSTMs, GRUs, reservoir computers

exploding gradient - if θ is too big, next iteration θ become even bigger

- * this can be solved
- 1. clip the gradient - if $\theta > C$, then $\theta > \theta C / \|\theta\|$
- 2. move to LSTMs, GRUs, reservoir computers

RNNs are not ideal for learning temporal correlations - **have a short-term memory**



p78–p79

different tasks * $T_h = T_y$ in table below

Type	Input time step	Label time step	Example
one-to-one	$T_x = 1$	$T_y = 1$	traditional NN
one-to-many	$T_x = 1$	$T_y > 1$	music generation
many-to-one	$T_x > 1$	$T_y = 1$	sentiment 观点 classification
many-to-many	$T_x = T_y$		name entity recognition
		$T_x \neq T_y$	machine translation

7 1.3

clustering 聚类 group the instances **based on their similarity** without labels

customer segmentation 细分, data analysis, dimensionality reduction * known as vector quantization, useful in **reduce order modelling**, anomaly detection, image segmentation

centroid - **k-means algorithm**

seek continuous regions of densely packed instances - **DBSCAN**

hierarchical (seek clusters of clusters) - **agglomerative 凝聚** and **divisive 分裂** algorithms

1.4

k-Means clustering centroid-based heuristic 启发式 algorithm

* finds a solution may not optimal but in good trade-off between optimality and time

find centroid μ_j to minimize $\frac{1}{km} \sum_{j=1}^k \sum_{i=1}^m \|x^{(i)} - \mu_j\|^2 = \text{minimize moment of inertia}$

* N_j is number of points in cluster j , N_j is unknown a priori, which is part of solution

1. choose a metric (distance function)

quantitatively defines the notion of **similarity** by measuring the distance between points in the feature space * choose Euclidean distance, $\|\cdot\|_2$ in this case

2. choose number of clusters k * significantly impact result, more clusters = less inertia

3. choose randomly the centroids μ_j , for $j = 1, \dots, k$ * run different initial., e.g. 10 times

4. compute the distances $\|x^{(i)} - \mu_j\|^2$, for $i = 1, \dots, m$ and $j = 1, \dots, k$

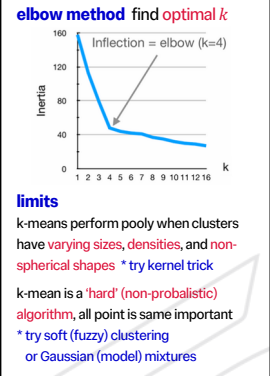
5. assign assign each instance $x^{(i)}$ to the closest centroid μ_j

6. compute the barycentre mean for each cluster $\mu_{j,\text{new}} = \frac{\sum_{i=1}^{N_j} x^{(i)}}{N_j}$, $z_j^{(i)} = 0$ or 1

7. update centroid overwrite centroid $\mu_j = \mu_{j,\text{new}}$

8. compute the new distances

9. repeat until $\mu_j - \mu_{j,\text{new}} \leq \text{tol}$ or reach maximum iterations * may only find local min.



1.5

DBSCAN density-based

- * density-based spatial clustering of application with noise

pro - do not need to specify k , can find cluster with complex shapes, can classify noise, and only need to tune ϵ and N

cons - can misclassify, heavily depend on distance metric, perform poorly with big densities difference, and challenge to tune ϵ if estimation on data scales and distance are not accurate

1. choose a metric (distance function)

- * choose Euclidean distance, $\|\cdot\|_2$

2. choose vicinity 邻近 ϵ

3. choose n. of point N point is need to classify a **neighborhood** as high density

4. select a new instance

5. count n. of instances around new instance satisfies $\|x^{(i)} - x^{(j)}\|^2 \leq \epsilon$

instance satisfies $\|x^{(i)} - x^{(j)}\|^2 \leq \epsilon$

if

n. of instances + 1 (new ins.) $\geq N$ in ϵ - neighborhood, new instance is a core instance

- * ϵ - neigh. may include other core ins.
- * all neighboring core ins. form a cluster

else

not a core region, return to 4

6. find anomalies classify as anomalies

- * e.g. noise if all ins. that are not core ins. and do not have a core ins. in their neigh.

1.6

hierarchical clustering

heuristic and greedy algorithm

- * greedy algorithm finds subproblem local optimum using heuristic algorithm, the solution of the problem at each stage is the input of next stage

bottom-up most common

1. choose a metric (distance function)

- * choose Euclidean distance, $\|\cdot\|_2$

2. find distance $\|x^{(i)} - x^{(j)}\|^2$ for all i, j

n. of dis. = $(m + 1 - \text{current iter.})^2$

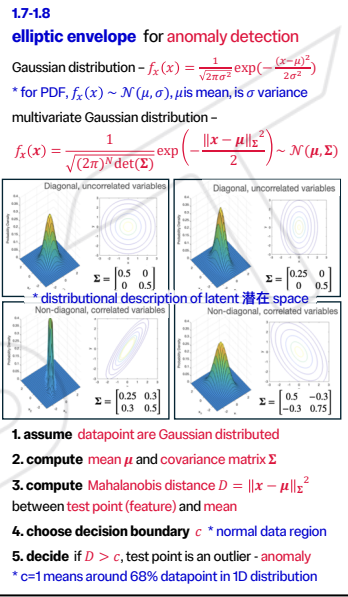
3. merge closet pair in a new cluster

- * criterion of merge is linkage criterion choose centroid linkage clustering
- * merged cluster midway of original ins.

4. repeat repeat 1. and 2. with $m - 1$ ins.

5. stop when have one large cluster

top-down known as divisive



8 2

PCA principal component analysis

principle variable - **linear combination of original variables** that captures the maximum variance

- * in 2D correlated case, prin. var. is linear comb. of 2 var.

principle axes - **direction of prin. var. in feature space**

- * all data input in prin. axes coordinates are uncorrelated

max $\text{Variance}_w = \frac{1}{N-1} \sum_{i=1}^N [(x^{(i)} - \bar{x}) \cdot w]^2$

subject to $\|w\|^2 = C$ * usually set constant $C = 1$

- * $\bar{x} = \sum_{i=1}^N x^{(i)} / N$ is empirical mean
- * normalize w to prevent its magnitude be variable

which has solution $\sum \bar{w}_i = \sum (w_i / \|w\|) = \lambda_i \bar{w}_i$

- * $i = 1, \dots, \text{rank}(\Sigma)$, eigenvalue in decreasing order
- * most and least important prin. axis are $\bar{w}_1$ and $\bar{w}_{\text{rank}}$
- * var. are math independent (not connected by a fun.)
- less sensitive to input noise compare to linear reg.

pro - **physical insight, and dimensionality reduction**

cons - **perform poorly with highly nonlinear data**

***autoencoder** deconstruct - reconstruct

denoising, image reconstruction, colorization, ...

- * encoder → bottleneck layer (latent space) → decoder

***autoencoder neural network**

loss - reconstruction loss in mse, mae, ...

***convolutional autoencoder**

model **highly nonlinear**, structured data (e.g. images) without assuming Gaussian distributions

*** transfer learning** re-train exist network

1. freeze N low-level layers, parameters will not update

2. modify output layers to have correct n. of output

- * can add preprocessing step to resize input as well

3. re-train & re-validate retrain output layers

4. unfreeze retrain more hidden layer if poor perform.

- * more training data, more unfreeze layers, reduce α

offline learning batch learning

online learning incremental learning

large α - quickly adapt to new data but forget old one